苏州大学

《新一代人工智能原理》大作业

——著名的 A*算法

学　　院：　　机电工程学院

班　　级：　　20 电气一班

学　　号：　　2029402016

姓　　名：　　苗　业

任课老师：　　陈　良

一、 $A*$算法简介

$A*$算法是一种启发式搜索方法，目前在网络路由算法、机器人、人工智能、游戏设计等方面有着普遍的应用。$A*$算法一般是以估价函数$f(n)$的大小来排列待扩展状态的次序，每次选择$f(n)$值最小者进行扩展。估价函数可以分为以下两个部分：

$$f(n) = g(n) + h(n)$$

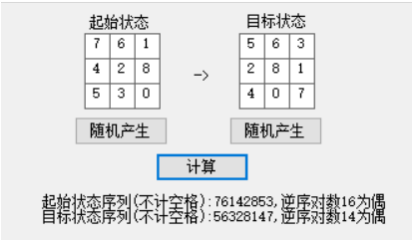
其中$g(n)$是初始结点到n结点的实际代价，而$h(n)$是从n结点到目的结点的最佳路径的估计代价，且满足$h(n) <= h^*(n), h^*(n)$为n结点到目的结点的最优路径的代价。

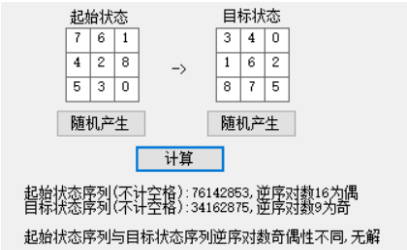
找到全局最优解的关键在于启发函数$h(n)$的选取：若$h(n)$小于等于n结点到目标结点最优路径的距离实际值，这种情况下，搜索的点数多，搜索范围大，效率低，但能得到全局最优解；若$h(n)$大于实际值，搜索的点数少，搜索范围小，效率高，但不能保证得到全局最优解。估价值与实际值越接近，估价函数取得就越好。

二、 八数码问题实验案例

使用实验软件演示程序时，软件将$closed$表与$open$表中的节点按步骤给出，并给出每一节点实际代价和估计代价的数值，搜索过程中对$open$表中的状态进行估价函数大小的排列，选择$f(n)$值最小的节点进行扩展，将其放入$closed$表，同时将其子节点加入$open$集中。重复这个过程，最终得到搜索结果与最优解，即为存放在$closed$表中的状态序列。(演示软件中所运用的估计代价函数为不在位的号码数量)

使用实验软件验证程序时，我们可以自定义八数码初始状态与目标状态，随后使用软件进行计算验证，可以看到，当起始状态与目标状态逆序对数奇偶性相同时，程序会给出深度优先搜索、宽度优先搜索以及$A*$算法在使用不同启发函数以及$open$表排列方式时算法效果的不同。而当两种状态逆序数量不同时，八数码问题无解，这一部分将在报告第四部分做出分析。





算法	启发函数h	OPEN表排序方式	搜索步数	访问结点数	耗时/毫秒
深度优先搜索	/	/	5521	5641	0
宽度优先搜索	/	/	25	154113	295
A*算法	不在位数	只按f排序	25	38395	100
A*算法	不在位数	先按f排序,后按h排序	25	33233	93
A*算法	哈密尔顿距离	只按f排序	25	3094	10
A*算法	哈密尔顿距离	先按f排序,后按h排序	25	2562	0

三、 使用其他启发函数解决八数码问题

在使用实验验证程序时，我注意到不同启发函数对算法的影响不同，以下以曼哈顿距离以及欧几里得距离为例进行八数码问题的求解：

哈密尔顿距离是指在n维空间中两个点之间的距离，它的定义是将每个维度上的坐标差的绝对值累加起来，即

$$d(x,y) = \Sigma\,|x_i - y_i|$$

其中x和y是两个向量，i是向量中的维度。哈密尔顿距离不考虑两个点之间的直线距离，而是考虑两个点在每个维度上的坐标差的绝对值之和。这种距离度量常用于分类、聚类等机器学习任务中。

曼哈顿距离是指在二维坐标系中从点A到点B沿着网格线走的最短距离，即是$n = 2$时的哈密尔顿距离，在八数码问题中可以得到应用。

欧几里得距离也是n维空间中两个点之间的距离表示方法，它的定义是将每个维度上的坐标差的平方求和再开方，即

$$d(x,y) = sqrt(\Sigma(x_i - y_i)\text{^}2)$$

与曼哈顿距离不同的是，欧几里得距离可以沿着任意方向行走，即可以斜着走。这种距离度量常用于数据挖掘、机器学习、图像处理等领域中。与其他距离度量相比，欧几里得距离常用于维度较少（一般不超过10维）的数据集。

利用两个函数分别定义这两种启发函数：

```python
def get_h_manhattan(node):
    h = 0
    for i in range(9):
        if node.state[i] == 0:
            continue
        h += abs(i % 3 - target.index(node.state[i]) % 3) + abs(i // 3 - target.index(node.state[i]) // 3)
    return h
```

```python
def get_h_Euclid(node):
    h = 0
    for i in range(9):
        if node.state[i] == 0:
            continue
        x1, y1 = i % 3, i // 3
        x2, y2 = target.index(node.state[i]) % 3, target.index(node.state[i]) // 3
        h += math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
    return h
```

在附件中我给出了运用这两种启发函数来解八数码程序的完整$python$代码，从各自的运行耗时也可以看出，不同的启发函数对于搜索算法也有不同的影响。

四、 对八数码是否有解问题的思考

在实验软件操作中可以得知，当起始状态与目标状态逆序对数奇偶性相同时，可以得出运算结果，而当起始状态与目标状态逆序对数奇偶性不同时，这类八数码问题是无解的。

其实这可以看作一个简单的数学问题，将由九个数（将空格看作 0）组成的$3*3$矩阵转化为一维的一行数据，那么我们在移动八数码中的空格时，可以将其转化为一维数组中元素的位置的改变。当我们进行空格的左右移动时，实质上一维数组的元素排列顺序没有改变，即不影响此八数码状态的逆序对数；而当我们进行空格的上下移动时，实际上是将一维数组中的一个元素移到相隔两个数据的位置上，这可以分为三种情况：

1. 当这三个元素原本就按照无逆序顺序排列时，逆序对数增加 2；
2. 当这三个元素原本按照逆序排列时，逆序对数减少 2；
3. 当这三种元素本来顺序有正有逆时，逆序对数不变。

因此，在经过空格的一系列移动后，末状态和初状态的逆序对数奇偶性必然相同，在这里我们可以利用之前学过的拒取式推理，即

拒取式推理

$$P \rightarrow Q,\ \neg Q,\ \Rightarrow \neg P$$

来证明，当起始状态与目标状态逆序对数奇偶性不同时，这类八数码问题是无解的。

因此，在程序中我加入了判断当前八数码问题是否有解的逆序对数计数函数：

```python
def reverse_count(state):
    inversion_count = 0
    for i in range(8):
        for j in range(i + 1, 9):
            if state[i] != 0 and state[j] != 0 and state[i] > state[j]:
                inversion_count += 1
    return inversion_count
```

经过查阅资料，我了解到这种判断逆序对数的方法还可以推广到$N*N$的数据结构中，不过要分类讨论，这里就不再赘述了。

五、 小结

在对搜索求解策略的学习中，我从基础的状态空间表示法开始，一步步深入学习到启发式搜索策略。而作为启发式搜索策略的经典算法，对$A*$算法的学习也让我对前面的知识进行了巩固，不仅如此，在以后的学习研究中，这次大作业促使我学习的算法编程知识也将长久地发挥作用，我觉得收获颇丰。